Análise Comparativa de Algoritmos de Busca Aplicados ao Problema Lights Out

Vinicius Barauce Leite Baena
Universidade Tuiuti do Paraná
Bacharelado em Ciência da Computação
vinicius.baena@aluno.utp.br

2026

Resumo

Este trabalho apresenta a implementação e análise comparativa de algoritmos de busca aplicados ao problema Lights Out. Foram avaliados algoritmos clássicos e heurísticos, incluindo BFS, DFS, Greedy, A*, Hill Climbing e Simulated Annealing. A análise considera tempo de execução, uso de memória, capacidade de encontrar solução e qualidade da solução. Os resultados demonstram diferenças significativas entre abordagens exatas e heurísticas, evidenciando o impacto da escala do problema no desempenho computacional.

1 Introdução

Problemas de busca são fundamentais na Ciência da Computação, sendo amplamente utilizados para encontrar sequências de ações que transformem um estado inicial em um estado objetivo [Russell and Norvig, 2021]. O problema Lights Out consiste num quebra-cabeça lógico onde o objetivo é apagar todas as luzes de um tabuleiro através de pressões em células específicas.

Este trabalho tem como objetivo comparar diferentes algoritmos de busca, analisando o seu desempenho e eficiência ao lidar com o crescimento do espaço de estados deste problema.

2 Descrição do Problema

O Lights Out é modelado num tabuleiro $N \times N$, onde cada célula possui dois estados possíveis: acesa (1) ou apagada (0). A dinâmica fundamental reside no facto de que, ao

pressionar uma célula, tanto ela como os seus vizinhos adjacentes (norte, sul, leste e oeste) sofrem uma inversão de estado. O objetivo final é alcançar a configuração onde todas as células estão no estado 0.

3 Metodologia

Os algoritmos foram implementados em linguagem Python e submetidos a testes em tabuleiros de dimensões crescentes: 2x2, 3x3, 5x5, 7x7 e 10x10. Foram selecionados os seguintes algoritmos para análise:

- **Busca Cega:** Busca em Largura (BFS) e Busca em Profundidade Limitada (DFS).
- **Busca Informada/Heurística:** Busca Gulosa (Greedy) e A*.
- **Busca Local:** Hill Climbing e Simulated Annealing.

Para os algoritmos informados, utilizou-se uma heurística baseada na contagem de luzes apagadas. A base teórica para a determinação de caminhos de custo mínimo através de heurísticas segue o modelo formal proposto por Hart et al. [1968]. Foram definidos limites de execução de 20 segundos por instância e um teto de memória de 150.000 estados explorados.

4 Resultados

A análise comparativa inicial dos algoritmos está sintetizada na Tabela 1, considerando as métricas gerais de desempenho e qualidade[cite: 62, 66].

Tabela 1: Comparação geral de desempenho dos algoritmos

Algoritmo	**Tempo**	**Memória**	**Sucesso**	**Ótimo**
BFS	Alto	Alto	100%	Sim
DFS	Médio	Baixo	Variável	Não
Greedy	Baixo	Baixo	Médio	Não
A*	Médio	Médio	Alto	Sim
Hill Climbing	Baixo	Baixo	Baixo	Não
Simulated Annealing	Médio	Baixo	Médio	Não

4.1 Escalabilidade

Observou-se que algoritmos de busca cega, como o BFS, apresentam crescimento exponencial de consumo de recursos [Russell and Norvig, 2021]. Em instâncias maiores (acima de 5x5), o BFS frequentemente atingiu o limite de memória ou tempo antes de encontrar

a solução. Em contrapartida, o A* manteve um equilíbrio superior entre a garantia de solução ótima e o consumo de recursos computacionais.

5 Discussão

A análise experimental evidencia o *trade-off* entre eficiência computacional e qualidade da solução[cite: 69]. Algoritmos exatos garantem a solução ótima, mas tornam-se inviáveis para grandes espaços de estados. A busca gulosa e o Hill Climbing são extremamente rápidos, porém suscetíveis a mínimos locais, o que resultou em falhas em configurações mais complexas do tabuleiro. O Simulated Annealing demonstrou maior robustez que o Hill Climbing simples, conseguindo escapar de alguns ótimos locais.

6 Conclusão

Os resultados demonstram que a escolha do algoritmo depende diretamente da escala do problema[cite: 71]. O BFS é eficaz apenas para problemas de baixa dimensionalidade, enquanto o A* se destaca como a opção mais equilibrada. Para problemas de escala massiva onde a optimalidade não é obrigatória, abordagens heurísticas e locais tornam-se indispensáveis.

Referências

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, 3 edition, 2009.

Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

Richard E. Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence*, 27(1):97–109, 1985.

Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, 4 edition, 2021.

A Código Fonte

Abaixo, apresenta-se o trecho principal da heurística utilizada, inspirada nas estruturas de busca exploradas por Korf [1985]:

```python
def heuristic(board):
    # Retorna a contagem de celulas apagadas
    return sum(cell == 0 for row in board for cell in row)
```